TABLE OF CONTENTS

S. No.	Topic	Page
1	Introduction to Object-Oriented Programming	2
2	Meaning and Definition of Polymorphism	3
3	Types of Polymorphism	4
4	Compile-Time Polymorphism	5
5	Function Overloading	7
6	Operator Overloading	8
7	Run-Time Polymorphism	9
8	Function Overriding and Virtual Functions	11
9	Case Study: Payment Management System	12
10	C++ Program and Explanation	13
11	Output, Testing, and Analysis	15
12	Advantages, Limitations, and Applications	16
13	Conclusion and References	17

Page numbers reflect this edited version.

ABSTRACT

Polymorphism is one of the major principles of object-oriented programming. The word means “many forms.” In C++, polymorphism allows the same function name or interface to produce different behaviour according to the arguments supplied or the actual object involved. This case study introduces polymorphism, explains its major forms, and demonstrates its use through a Payment Management System that supports different payment methods.

1. INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING

Object-Oriented Programming (OOP) is a programming approach in which software is designed using classes and objects. A class acts as a blueprint, while an object is an instance of that class. OOP helps programmers organise code into logical units and makes larger programs easier to maintain.

1.1 Main principles of OOP

- Encapsulation: Combining data and the functions that operate on it inside a class, while controlling access to the data.
- Abstraction: Showing essential features and hiding unnecessary implementation details.
- Inheritance: Creating a new class from an existing class so that suitable properties and behaviour can be reused.
- Polymorphism: Allowing one interface, function name, or common operation to work in different ways.

1.2 Why OOP is useful

OOP is useful when a program contains many related entities. For example, a college system may contain Student, Teacher, Course, and Examination classes. Each class can store its own data and define suitable operations. Inheritance can represent shared features, while polymorphism can provide different behaviour for related objects.

1.3 Role of C++

C++ supports procedural programming as well as object-oriented programming. It provides classes, objects, inheritance, access specifiers, constructors, function overloading, operator overloading, virtual functions, and abstract classes. These features make C++ suitable for learning how polymorphism works.

Polymorphism is particularly useful when different objects share a common interface but must perform an operation differently. A payment system, for instance, can process UPI, card, and cash payments through a common payment interface.

2. MEANING AND DEFINITION OF POLYMORPHISM

The term polymorphism is formed from two Greek words: “poly,” meaning many, and “morph,” meaning forms. In programming, polymorphism means that a common name or interface can represent more than one behaviour.

In C++, polymorphism allows the same operation to behave differently depending on the arguments or the object on which it is performed. It improves flexibility and supports reusable program design.

2.1 Simple example from daily life

Consider the word “pay.” A person may pay using cash, a debit card, or UPI. The purpose is the same—completing a payment—but the steps differ according to the payment method. A well-designed program can use a common payment operation and allow each payment type to implement its own steps.

2.2 Important terms

- Interface: The common operation available to the user, such as processPayment().
- Overloading: Defining multiple functions with the same name but different parameter lists.
- Overriding: Providing a derived-class implementation of an inherited virtual function.
- Binding: Connecting a function call to the function implementation that will run.
- Static binding: The choice is made at compile time.
- Dynamic binding: The implementation is selected at runtime based on the actual object.

2.3 Purpose of polymorphism

The main purpose is to let programmers write general code that can work with different types of objects. Rather than writing separate high-level logic for every payment type, the program can call a common function. Each payment class then handles its own details.

Polymorphism does not mean that every function with the same name is interchangeable. C++ follows specific rules for overload resolution and virtual dispatch. Correct function signatures and appropriate use of virtual functions are important.

3. TYPES OF POLYMORPHISM

In introductory C++ courses, polymorphism is commonly divided into two main categories: compile-time polymorphism and run-time polymorphism.

3.1 Compile-time (static) polymorphism

In compile-time polymorphism, the compiler determines which function or operation should be used before the program runs. Function overloading and operator overloading are common examples. The selected function depends on information such as the number and types of arguments.

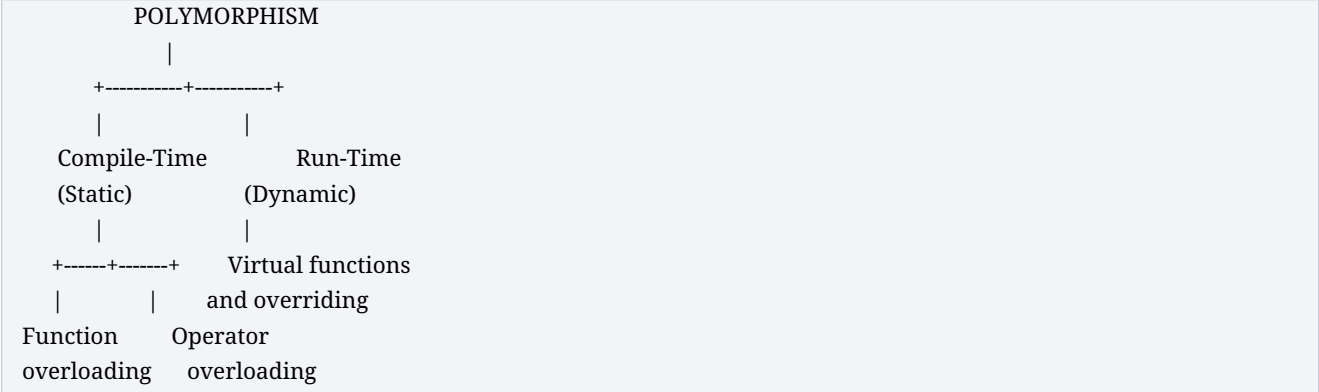
3.2 Run-time (dynamic) polymorphism

In run-time polymorphism, the program selects an overridden virtual function based on the actual object involved in a call through a base-class pointer or reference. It is commonly implemented through inheritance and virtual functions.

3.3 Comparison

Feature	Compile-Time	Run-Time
Decision	Made by compiler	Made during execution
Common mechanism	Function/operator overloading	Virtual function overriding
Inheritance required?	Not necessarily	Usually used with inheritance
Flexibility	Based on argument list	Based on actual object type
Typical example	add(2, 3) or add(2.5, 3.5)	Base pointer calling a derived virtual function

3.4 General classification diagram



The classification is a useful learning model. C++ also supports other forms of polymorphism, such as templates, but this report focuses on the standard introductory categories.

4. COMPILE-TIME POLYMORPHISM

Compile-time polymorphism allows the same function name or operator to be used in more than one valid way. The compiler chooses the suitable version based on the information available in the program.

4.1 Characteristics

- The choice of overload is made during compilation.
- It can make program code easier to read by using one meaningful name for related operations.
- Function overloading depends on different parameter lists.
- Operator overloading gives selected operators a suitable meaning for user-defined types.
- An overload cannot be created merely by changing the return type.

4.2 Example: Function overloading

```
#include <iostream>
using namespace std;

int add(int a, int b) {
   return a + b;
}

double add(double a, double b) {
   return a + b;
}

int main() {
   cout << add(4, 6) << endl;
   cout << add(2.5, 3.5) << endl;
   return 0;
}
```

Expected output

```
10
6
```

4.3 Explanation

The program defines two functions named add(). The first accepts two integers, while the second accepts two double values. When add(4, 6) is called, the integer version is

selected. When add(2.5, 3.5) is called, the double version is selected. The same function name therefore supports different argument types.

4.4 Common mistake

The declarations int calculate(int x) and double calculate(int x) cannot be overloaded based only on their return types. The parameter lists must differ in a way that allows the compiler to distinguish the calls.

5. FUNCTION OVERLOADING

Function overloading means defining two or more functions with the same name in the same scope but with different parameter lists. It is a common example of compile-time polymorphism.

5.1 Ways to overload a function

- Different number of parameters: display(int) and display(int, int).
- Different parameter types: display(int) and display(string).
- Different parameter order or combinations: show(int, double) and show(double, int), provided calls are unambiguous.

5.2 Program: Area calculator

```
#include <iostream>
using namespace std;

int area(int side) {
   return side * side;
}

int area(int length, int breadth) {
   return length * breadth;
}

double area(double radius) {
   return 3.14 * radius * radius;
}

int main() {
   cout << "Square area: " << area(4) << endl;
   cout << "Rectangle area: " << area(4, 5) << endl;
   cout << "Circle area: " << area(2.0) << endl;
   return 0;
}
```

Expected output

```
Square area: 16
Rectangle area: 20
Circle area: 12.56
```

6. OPERATOR OVERLOADING

Operator overloading allows selected C++ operators to work with user-defined types, such as objects of a class. For example, the + operator can be defined to add two ComplexNumber objects. Operator overloading is another form of compile-time polymorphism.

6.1 Why it is used

Operators such as +, -, ==, and < have familiar meanings for built-in types. With operator overloading, programmers can define a logical meaning for selected operators when working with class objects. This can make expressions more natural.

6.2 Example: Adding two complex numbers

```
#include <iostream>
using namespace std;

class Complex {
public:
   int real, imag;

   Complex(int r = 0, int i = 0) {
      real = r;
      imag = i;
   }

   Complex operator+(const Complex& other) {
      return Complex(real + other.real,
                     imag + other.imag);
   }

   void display() {
      cout << real << " + " << imag << "i" << endl;
   }
};

int main() {
   Complex a(2, 3), b(4, 5);
   Complex c = a + b;
   c.display();
   return 0;
}
```

Expected output

```
6 + 8i
```

7. RUN-TIME POLYMORPHISM

Run-time polymorphism allows a program to call the appropriate overridden function based on the actual type of an object during execution. In C++, it is commonly achieved by declaring a base-class member function virtual and overriding it in derived classes.

7.1 How it works

1. A base class declares a virtual function.
2. One or more derived classes provide their own implementations of that function.
3. A base-class pointer or reference refers to an object of a derived class.
4. When the virtual function is called, the derived class implementation is selected for that actual object.

7.2 Example

```
#include <iostream>
using namespace std;

class Animal {
public:
   virtual void sound() {
      cout << "Animal makes a sound" << endl;
   }
};

class Dog : public Animal {
public:
   void sound() override {
      cout << "Dog barks" << endl;
   }
};

int main() {
   Dog d;
   Animal* ptr = &d;
   ptr->sound();
   return 0;
}
```

Expected output

```
Dog barks
```

7.3 Explanation

The pointer ptr is declared as Animal*, but it points to a Dog object. Because sound() is virtual, the program calls Dog::sound() rather than Animal::sound(). This is dynamic dispatch: the implementation is chosen using the actual object type.

7.4 Important requirement

The base function must be virtual for this form of runtime dispatch. If sound() were non-virtual, calling it through an Animal pointer would use the base-class function. In real projects, a polymorphic base class should generally have a virtual destructor when objects may be deleted through a base-class pointer.

8. FUNCTION OVERRIDING AND VIRTUAL FUNCTIONS

8.1 Function overriding

Function overriding occurs when a derived class provides its own implementation of an inherited virtual member function with a matching signature. It lets a derived class customise inherited behaviour while retaining a common interface.

8.2 Virtual function

A virtual function is a non-static member function declared with the keyword virtual in a base class. Calls made through base-class pointers or references can then invoke the derived implementation when the object belongs to a derived class.

8.3 The override keyword

The override specifier, introduced in modern C++, tells the compiler that the programmer intends to override a base-class virtual function. If the signature does not correctly override a base function, the compiler reports an error. This helps prevent accidental mistakes.

8.4 Pure virtual function and abstract class

```
class Payment {
public:
   virtual void pay(double amount) = 0;
   virtual ~Payment() = default;
};
```

A pure virtual function is declared using = 0. A class with a pure virtual function is abstract and cannot normally be instantiated directly. Derived classes must implement the pure virtual function to become concrete, unless they too remain abstract.

8.5 Overloading versus overriding

Point	Overloading	Overriding
Meaning	Same name, different parameter lists	Derived class replaces inherited virtual behaviour
Classes	May occur in one class or scope	Requires inheritance
Selection	Compile time	Run time for virtual calls
Purpose	Related operations for different inputs	Different behaviour for different object types

9. CASE STUDY: PAYMENT MANAGEMENT SYSTEM

9.1 Problem statement

A small online shopping application needs to accept different payment methods, such as UPI, card, and cash. If the application uses a separate set of instructions throughout the program for every payment type, the code can become repetitive and difficult to maintain. Adding a new payment option may require changing many parts of the program.

9.2 Proposed solution

The proposed solution uses a common abstract class named Payment. It declares a common operation processPayment(). Separate derived classes—UPIPayment, CardPayment, and CashPayment—implement the operation according to their own payment methods. The main program stores references to these payment objects through a common base-class pointer and calls the same operation.

9.3 Objectives

- Demonstrate runtime polymorphism using a practical example.
- Use inheritance to share a common payment interface.
- Apply virtual functions and function overriding.
- Reduce repeated high-level code in the main program.
- Make the design easier to extend with additional payment types.

9.4 Scope and assumptions

This is an educational console-based demonstration. It does not connect to a bank, card network, UPI service, or payment gateway. The payment messages only simulate the outcome. Real payment software would need secure authentication, transaction verification, error handling, logging, encryption, and compliance with applicable requirements.

9.5 Requirements

- Language: C++
- Concepts: classes, inheritance, pointers, virtual functions, overriding, vectors
- Software: any C++ compiler or IDE, such as GCC, Visual Studio, or Code::Blocks
- Input: a fixed transaction amount in the demonstration
- Output: a method-specific payment message

10. COMPLETE C++ IMPLEMENTATION

The following program demonstrates runtime polymorphism. The Payment class defines the common interface. Each derived class overrides processPayment(). The main function uses a vector of base-class pointers so that the same call can produce different results.

```
#include <iostream>
#include <vector>
using namespace std;

class Payment {
public:
   virtual void processPayment(double amount) = 0;
   virtual ~Payment() = default;
};

class UPIPayment : public Payment {
public:
   void processPayment(double amount) override {
      cout << "UPI payment of Rs. "
         << amount << " processed." << endl;
   }
};

class CardPayment : public Payment {
public:
   void processPayment(double amount) override {
      cout << "Card payment of Rs. "
         << amount << " processed." << endl;
   }
};

class CashPayment : public Payment {
public:
   void processPayment(double amount) override {
      cout << "Cash payment of Rs. "
         << amount << " received." << endl;
   }
};

int main() {
   UPIPayment upi;
   CardPayment card;
   CashPayment cash;

   vector<Payment*> payments = {&upi, &card, &cash};
   double amount = 1500.00;
```

```
    for (Payment* payment : payments) {
        payment->processPayment(amount);
    }

    return 0;
}
```

10.1 Explanation of the program

- #include <iostream> enables console input/output, while <vector> provides a container for the base-class pointers.
- Payment is an abstract class because processPayment() is a pure virtual function.
- UPIPayment, CardPayment, and CashPayment inherit from Payment.
- The override keyword checks that each derived function correctly overrides the base virtual function.
- The vector stores pointers of type Payment* that refer to objects of different derived classes.
- The loop calls processPayment() on each pointer. Runtime polymorphism selects the matching derived implementation.

The virtual destructor is included so that the base class is suitable for polymorphic use and can safely support deletion through a base pointer in an expanded version of the program.

11. OUTPUT, TESTING, AND ANALYSIS

11.1 Expected output

```
UPI payment of Rs. 1500 processed.
Card payment of Rs. 1500 processed.
Cash payment of Rs. 1500 received.
```

The output is expected when the program is compiled and run successfully. The exact formatting of decimal values may vary if the program is changed to set a specific number of decimal places.

11.2 Test cases

Test	Object	Expected behaviour	Result
1	UPIPayment	Displays UPI payment message	Pass if message matches
2	CardPayment	Displays card payment message	Pass if message matches
3	CashPayment	Displays cash received message	Pass if message matches
4	All three	Same base pointer type calls three implementations	Pass if all differ correctly

11.3 Analysis

The case study demonstrates that the program does not need a different loop for every payment method. All payment objects are handled through the Payment interface. The derived classes provide the specialised behaviour. This reduces repetition and makes the design easier to extend.

11.4 Extending the system

A future version could add WalletPayment or NetBankingPayment by creating a new class derived from Payment and implementing processPayment(). The main design can remain largely unchanged if the new object is added to the collection. This is an example of extensibility.

11.5 Limitations of the prototype

The program uses a fixed amount and prints simulated messages. It does not validate amounts, request user choices, check payment status with a real service, or handle network failures. These features would be required in a production application.

12. ADVANTAGES, LIMITATIONS, AND APPLICATIONS

12.1 Advantages of polymorphism

- Code reusability: A common interface can be reused for several related classes.
- Flexibility: Different derived classes can provide their own implementation.
- Maintainability: Changes to one implementation may not require changes to all calling code.
- Extensibility: New types can be added with limited changes to code that uses the common interface.
- Readability: Common operations can be expressed through one meaningful function name.
- Loose coupling: The main program can depend on an interface rather than every concrete class.

12.2 Limitations and challenges

- Virtual dispatch and object-oriented designs can make program flow less obvious to beginners.
- Runtime polymorphism may involve small runtime and memory overhead, depending on implementation.
- Incorrect signatures can prevent an intended override; using override helps detect this.
- Too many unnecessary classes can make a small program more complex than required.
- Testing must cover each derived implementation and relevant error conditions.
- Polymorphism alone does not provide security, validation, or correctness; these must be designed separately.

12.3 Applications in software development

- Payment systems: UPI, card, cash, wallet, and net banking methods.
- Graphics applications: different shapes implement a common draw() operation.
- Education systems: student categories may calculate or display information differently.
- Games: different characters may implement attack(), move(), or defend() in their own way.
- Media software: audio and video files can share a common play() interface.
- Data-processing systems: different data sources can implement a shared read() operation.

13. CONCLUSION AND REFERENCES

13.1 Conclusion

Polymorphism is an important principle of object-oriented programming because it allows a common name or interface to support different behaviours. In C++, compile-time polymorphism is demonstrated through function overloading and operator overloading, while runtime polymorphism is commonly implemented using inheritance, virtual functions, and overriding.

The Payment Management System shows how a single processPayment() interface can support UPI, card, and cash payment classes. The main program can call the same function on different objects and receive method-specific behaviour. This improves flexibility and makes it easier to extend the program with new payment types.

The case study also shows that good software design requires more than using polymorphism. A real payment application would need input validation, secure integration with payment services, transaction verification, logging, and error handling. Nevertheless, the example provides a clear foundation for understanding how polymorphism can help organise a program.

13.2 Learning outcomes

- Define polymorphism and explain why it is used in OOP.
- Differentiate compile-time and runtime polymorphism.
- Write examples of function overloading and operator overloading.
- Explain overriding, virtual functions, pure virtual functions, and abstract classes.
- Apply runtime polymorphism in a simple C++ application.
- Identify benefits and limitations of a polymorphic design.

13.3 References

5. Microsoft Learn, “Function Overloading.” https://learn.microsoft.com/en-us/cpp/cpp/function-overloading
6. Microsoft Learn, “Virtual Functions.” https://learn.microsoft.com/en-us/cpp/cpp/virtual-functions
7. cppreference.com, “virtual function specifier.” https://en.cppreference.com/w/cpp/language/virtual
8. cppreference.com, “Overload resolution.” https://en.cppreference.com/w/cpp/language/overload_resolution